@aman_views

Gen AI API Integration

From Basics to Building Real Integrations — using the Claude API as our working example.

What this handbook covers

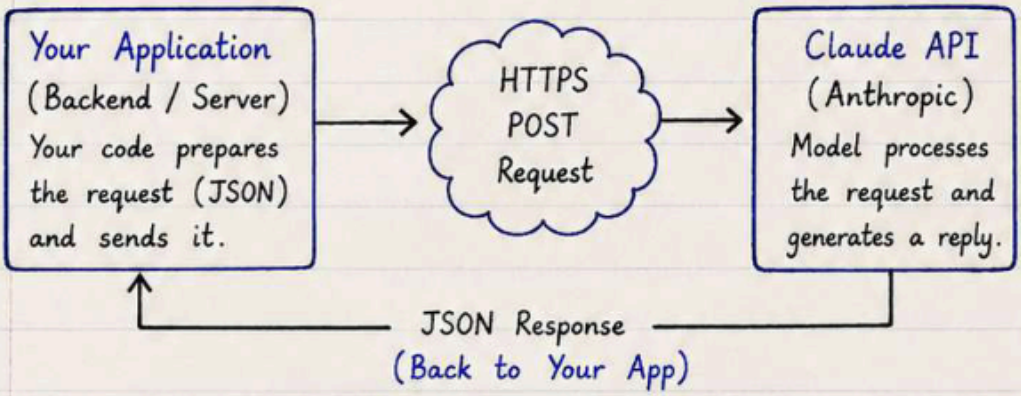
- → How AI APIs work under the hood — request, response, tokens
- → Getting and securing an API key the right way
- → Reading and writing the request/response JSON
- → System prompts, temperature, top_p/top_k — controlling output
- → Multi-turn conversations & streaming responses
- → Tool use (function calling), vision input, and other built-in tools
- → Error handling, rate limits, and production best practices

Why this matters ?

APIs let developers bring AI power directly into their apps, automate workflows, and create intelligent experiences at scale.

You build once, AI works for you forever.

How it all fits together



Remember :

The API doesn't remember anything by itself. You need to send the full conversation history every time.

Key things to know

- Communication happens over HTTPS using REST API.
- Requests are in JSON format. Responses are also in JSON.
- You authenticate using an API key in headers.
- Every request is independent (stateless).
- You pay for tokens (input + output). More tokens = more cost.
- The better your prompt + parameters, the better the output.

Goal of this Handbook :

By the end of this guide, you will be able to confidently integrate Claude API into your projects, handle real-world cases, and build production-ready AI features.

1. What Is an AI API?

REST basics — and why you integrate instead of just using the chat app

An AI API lets your code talk to a model like Claude directly — no browser, no chat window. Your backend sends a request over HTTPS, the model processes it, and sends back a structured JSON response. This is what powers chatbots, automated content tools, coding assistants, and support systems.

Why integrate via API instead of the chat UI?

- Automation — no human needs to click 'send' each time
- Scale — handle thousands of requests/day programmatically
- Control — set exact parameters (tone, length, creativity)
- Embed AI inside your own product / website / app

How it fits together



Tip: the API is stateless — Claude doesn't 'remember' past calls unless you resend the conversation history each time.

Extra things to know

- AI models are probabilistic — same input can give slightly different outputs each time.
- Token limits exist (input + output). You pay for the tokens you use.
- Keep requests idempotent — don't rely on side effects.
- Always use HTTPS; never send API keys in query params.
- Check the status codes. 200 = success, 4xx = your fault, 5xx = server fault.

Simple example (Python)

```
import requests
url = "https://api.anthropic.com/v1/messages"
headers = {"x-api-key": "sk-ant-xxxxxxxxxxxxxxx",
           "anthropic-version": "2023-06-01",
           "content-type": "application/json"}
payload = {"model": "claude-sonnet-5", "max_tokens": 300,
           "messages": [{"role": "user", "content": "Explain REST APIs in one line"}]}
response = requests.post(url, headers=headers, json=payload)
print(response.json()["content"][0]["text"])
```

This is the basic idea — in the next pages we'll go deeper into each part.

@aman_views

2. Getting Your API Key

Anthropic Console setup + how to store keys safely

Every API request needs authentication. Anthropic uses an API key, generated from the Console, and it must be sent with every request in a header.

Keep in Mind

- → One key = full access to your account
- → Treat it like a password
- → Rotate keys if leaked
- → Monitor usage in Console

Steps

1. Create an account on the Anthropic Console (console.anthropic.com)
2. Go to 'API Keys' and generate a new key
3. Add billing / usage limits if using paid tier
4. Copy the key immediately — it's shown only once
5. Store it securely (environment variables, secret managers, .env file)

Never hardcode your key

```
# BAD — never commit this to GitHub
api_key = "sk-ant-xxxxxxxxxxxxxxx"
```

⇒

```
# GOOD — load from environment variable
import os
api_key = os.environ.get("ANTHROPIC_API_KEY")
```

In a Jupyter notebook, keep the key in a .env file (never in the notebook cell itself), and load it with python-dotenv.

.env file (in project root)

```
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxx
```

→

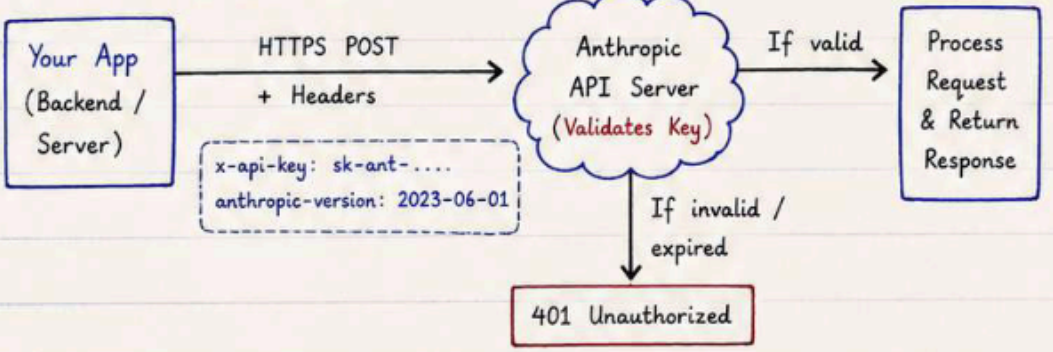
Load in Python

```
from dotenx import load_dotenv
load_dotenv()   # reads .env file
import os
api_key = os.getenv("ANTHROPIC_API_KEY")
```

Add .env to your .gitignore so it never gets pushed to GitHub.

How authentication works

You send your API key in the request header. The server validates it before processing your request.



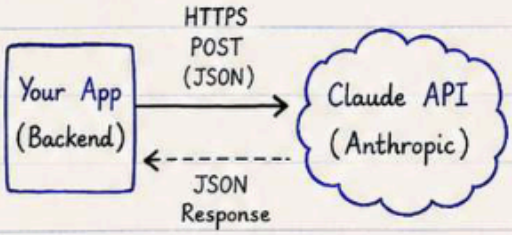
Pro Tip

Use environment variables in all environments (local, staging, production).

3. Your First API Call

Endpoint, headers, and the two ways to call it

The core endpoint is a single POST request. You can call it with raw HTTP (curl) or use Anthropic's official SDK (pip install anthropic) which handles headers, retries, and timeouts for you.



Option A — Raw HTTP (curl)

```
curl https://api.anthropic.com/v1/messages \
    -H "x-api-key: $ANTHROPIC_API_KEY" \
    -H "anthropic-version: 2023-06-01" \
    -H "content-type: application/json" \
    -d '{
        "model": "claude-sonnet-5",
        "max_tokens": 300,
        "messages": [
          {"role": "user", "content": "Explain REST APIs in one line"}
        ]
    }'
```

Header notes

- x-api-key → your secret key
- anthropic-version → API version
- content-type → we are sending JSON body
- Always use HTTPS (secure)

Option B — Python SDK (Recommended)

Easier to use in notebooks and production apps.

```
import anthropic
client = anthropic.Anthropic()  # reads key from environment variable
response = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=300,
    messages=[
        {"role": "user", "content": "Explain REST APIs in one line"}
    ]
)
print(response.content[0].text)
```

SDK Advantages

- ✓ Handles headers & auth for you
- ✓ Built-in retries
- ✓ Better error messages
- ✓ Cleaner code
- ✓ Works great in notebooks

A few extra things to know

→ This is a stateless call — every request is independent.

→ If you want conversation memory, send the full history in "messages".

→ max_tokens is the maximum tokens in the response (not exact length).

→ Choose the right model for your use case (speed vs quality vs cost).

Which model to use?

Haiku	→	Fast + Cheap
Sonnet	→	Balanced (best overall)
Opus	→	Highest quality (costly)

Quick Tip

Start small → test in curl → move to SDK → wrap in functions → add error handling.

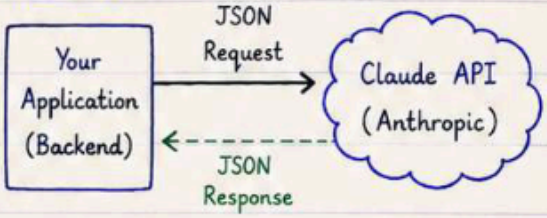
@aman_views

4. Request JSON — Anatomy

@aman_views

What you send: model, messages, roles, max_tokens, system, and more

Every request you send to Claude API is a JSON object. Small changes in parameters can lead to very different outputs. Let's break down each field.



Example Request

```
{
  "model": "claude-sonnet-5",
  "max_tokens": 500,
  "temperature": 0.7,
  "top_p": 0.95,
  "system": "You are a helpful SDET interview coach.",
  "messages": [
    { "role": "user", "content": "What is a flaky test?"}
  ]
}
```

Field by Field Breakdown

Field	Description
model	Which Claude model will process the request.
max_tokens	Maximum number of tokens in the reply. Hard cap on output length.
system	Instructions that set the behavior, tone and context for Claude.
messages	The conversation so far as a list of role + content objects.
role	Either "user" (you) or "assistant" (Claude's previous reply).
content	The actual text (or content blocks) for that turn.
temperature	Controls randomness. Lower = more focused, higher = more creative.
top_p / top_k	Controls diversity of output (nucleus sampling).

Important Notes

- → The API is stateless → Claude doesn't remember anything by itself. You must send full conversation history in "messages" every time.
- → Order matters → messages are processed in the order you send them.
- → Keep prompts clear and specific → better input = better output.
- → Use system prompt to set role, tone, rules, format etc.
- → Don't forget max_tokens → no response will exceed this limit.

Another Example (with conversation history)

```
{
  "model": "claude-sonnet-5",
  "max_tokens": 600,
  "system": "You are a senior QA mentor.",
  "messages": [
    { "role": "user", "content": "What is unit testing?"},
    { "role": "assistant", "content": "Unit testing is ..."},
    { "role": "user", "content": "Give example in Java."}
  ]
}
```

Parameter Effects (Quick Guide)

temperature	0 (focused) — 1 (creative)	Higher = more random & creative
top_p	0.1 (narrow) — 1.0 (wide)	Lower = narrow range of words
max_tokens	short — long	More tokens = longer responses

Pro Tip: Start simple → one user message. Then add system prompt, then tweak temperature/top_p.

Log your requests & responses while building so you can debug and improve.

5. Response JSON — Anatomy

@aman_views

What you get back: content blocks, stop_reason, token usage

Claude replies with a JSON object. Besides the answer text, it also tells you why it stopped and how many tokens it used. Use this info for logging, analytics, cost tracking, and handling edge cases.

```
{
  "id": "msg_01AbCDefG...",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "REST APIs let systems talk over HTTP."
    }
  ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 23,
    "output_tokens": 15
  }
}
```

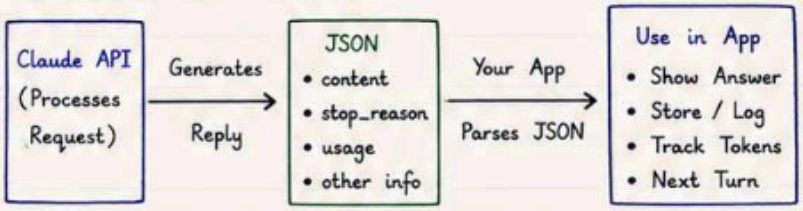
Field by Field Breakdown

Field	Meaning
id	Unique ID of this message.
type	Always "message" for normal replies.
role	"assistant" for Claude's reply.
content	Array of content blocks (text, tool_use, image, etc.).
stop_reason	Why Claude stopped generating. Examples: end_turn, max_tokens, stop_sequence, tool_use, pause_turn.
stop_sequence	If a stop sequence you set was hit, it will be shown here.
usage	Token usage for this response only. Helps with cost & quota tracking.

Common stop_reason values

- end_turn → Claude finished naturally.
- max_tokens → Reached the max_tokens limit.
- stop_sequence → Your stop sequence was found.
- tool_use → Claude decided to call a tool.
- pause_turn → Claude needs more input from you.

How the response flows back



Example: Minimal Python Handling

```
import anthropic, os
client = anthropic.Anthropic()
response = client.messages.create(
    model = "claude-sonnet-5",
    max_tokens = 300,
    messages = {"role": "user", "content": "Explain REST APIs in one line"}}
)
# Extract the answer text
answer = response.content[0].text
print("Answer:", answer)
# Log token usage
print("Tokens used - input:", response.usage.input_tokens,
        " | output:", response.usage.output_tokens)
print("Stop reason:", response.stop_reason)
```

Things to keep in mind

→ content can have multiple blocks (text, tool_use, thinking, etc.).

→ Always check stop_reason before assuming the answer is complete.

→ Track usage to estimate cost and avoid quota limits.

→ If stop_reason == "max_tokens", you may need to increase max_tokens or continue the conversation.

→ Don't rely on exact token counts — they are model-dependent estimates.

Quick Tip

Log every response (id, stop_reason, usage) during development.
It will save you hours when debugging weird outputs or cost spikes! 😊

6. Handling Errors & Retries

@aman_views

APIs fail. Good apps handle it smartly.

Common HTTP Status Codes

Code	Meaning	When it happens
200	OK	Success
400	Bad Request	Invalid JSON, missing field
401	Unauthorized	Invalid or missing API key
429	Too Many Requests	Rate limit exceeded
500	Server Error	Claude is having issues
529	Overloaded	Claude is overloaded

Best Practices

→ Always check status code before using the response.

→ Give clear error message to the user.

→ Use retries for 5xx errors or 429 (rate limit).

→ Don't retry for 4xx errors (your request is wrong).

→ Use exponential backoff for retries.

Exponential Backoff Example

Retry 1 → wait 1 second
Retry 2 → wait 2 seconds
Retry 3 → wait 4 seconds
Retry 4 → wait 8 seconds
... and so on (with a max limit)

Python: Simple Retry Logic (using requests)

```python
import time
import requests

url = "https://api.anthropic.com/v1/messages"
headers = {"x-api-key": ANTHROPIC_API_KEY,
           "anthropic-version": "2023-06-01",
           "contnrat-type": "application/json"}
payload = {...}

max_retries = 3
delay = 1
for attempt in range(max_retries):
    response = requests.post(url, headers=headers, json=payload)
    if response.status_code == 200:
        break  # success
    elif response.status_code in [500, 529, 429]:
        print(f"Retry {attempt+1} after {delay}s ...")
        time.sleep(delay)
        delay *= 2  # exponential backoff
    else:
        print(f"Failed with status {response.status_code}: {response.text}")
        break
else:
    print("Failed after all retries")
```

Error Response from Anthropic

```
{
  "type": "error",
  "error": {
    "type": "invalid_request_error",
    "message": "model is required",
    "param": "model",
    "code": "invalid_request_error"
  }
}
```

What to do?

- Show user-friendly message.
- Log the full error (for debugging).
- For rate limits (429), wait & retry.
- For invalid requests (400), fix your input.
- For 5xx, retry with backoff.

Pro Tips

✓ Log request id (if any) for support.

✓ Never expose your API key in frontend.

✓ Set timeout (connect + read).

✓ Handle network errors (no internet, DNS fail).

✓ Keep retry count small (2-4 is enough).

Example: Add timeout

```
response = requests.post(url, headers=headers,
           json=payload, timeout = (5, 30))
```

(5 sec to connect, 30 sec to read response)

Adjust based on your app needs.

7. Real World Example – Build a Mini Assistant

@aman_views

Let's build a simple CLI assistant that talks to Claude and prints the reply.

Flow

① User types a question
↓
② We send it to Claude API
↓
③ Claude processes and replies
↓
④ We print the answer

```python
import anthropic
import os

# Initialize client
client = anthropic.Anthropic(
    api_key=os.getenv("ANTHROPIC_API_KEY")
)

def ask_claude(question):
    response = client.messages.create(
        model="claude-3-5-sonnet-20241022",
        max_tokens=300,
        messages=[{"role":"user","content":question}]
    )
    return response.content[0].text

if __name__ == "__main__":
    while True:
        q = input("\nYou: ")
        if q.lower() in ["exit","quit"]:
            print("Bye!")
            break
        ans = ask_claude(q)
        print("\nClaude:", ans)
```

What this does?

- Reads user input from terminal
- Sends it to Claude
- Prints the response
- Repeat until user types exit/quit
- Super simple, but powerful!

Sample Conversation

You: Explain recursion in one line.
Claude: Recursion is when a function calls itself to solve a smaller version of the same problem.
You: Give example.
Claude: Factorial(n) = n * Factorial(n-1) ... until n == 0.
You: exit
Bye!

Key Takeaways

- ✓ You don't need to host any model.
- ✓ Just send good prompts, get good answers.
- ✓ Control cost with max_tokens.
- ✓ Use system prompt to guide behavior.
- ✓ Handle errors, retries and limits.
- ✓ Log everything!

Simple API call → Smart Assistant

Common Use Cases

Use Case	How Gen AI Helps	Example Prompt
Code Assistant	Write, explain, refactor code	"Explain this code: n [paste code]"
Q&A Bot	Answer questions from docs	"Answer from this doc: n [paste doc/extract]"
Summarization	Summarize long text	"Summarize this in 5 points: n [text]"
Classification	Categorize or label text	"Classify this feedback as Positive/Negative: [text]"
Content Generation	Create blogs, emails, replies	"Write a polite email to cancel a meeting."

Bonus Tip

Start small → one feature at a time.
Measure quality, latency, and cost → then scale.
Great prompts + clear context = Magic ✨

- ✓ Be specific
- ✓ Give examples
- ✓ Tell what format you want
- ✓ Provide context

8. Advanced Topics & Best Practices

@aman_views

Level up your integration and production readiness.

Streaming Responses

For long answers, use streaming to get tokens as they are generated.

```
{
  "model": "claude-3-5-sonnet-20241022",
  "messages": [
    { "role": "user",
      "content": "Write a poem" }
  ],
  "stream": true,
}
```

Response will be a series of events (chunks).
Assemble them to get full answer.

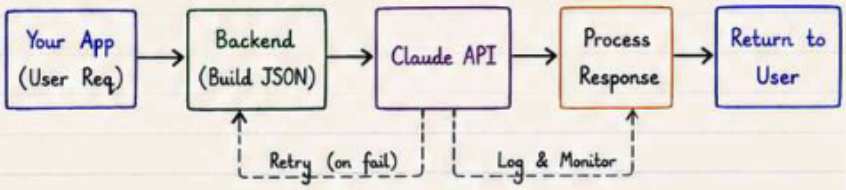
System Prompt Tips

- Be specific about role, tone, and format.
- Mention what NOT to do.
- Add examples in the prompt (few-shot).
- Keep instructions at top.
- Use placeholders for dynamic data.

Example

"You are a helpful coding assistant.
Explain step by step.
If unsure, say you don't know."

Typical Integration Flow (Production)



Rate Limits (Example)

Plan	Requests / Min	Tokens / Min	Notes
Claude Free	~20	~40K	Lower limits
Claude Pro	~50	~80K	Higher limits
Claude Team	~100+	~200K+	Depends on usage

Limits are per organization and can change.
Always handle 429 errors gracefully.

Observability & Monitoring

- ✓ Log request id, model, tokens, latency, status.
- ✓ Track token usage to control cost.
- ✓ Monitor errors (4xx / 5xx / timeouts).
- ✓ Set alerts for high failure rate or latency.
- ✓ Dashboard: cost trend, usage trend, top errors.

Common Pitfalls

- ✗ Sending too much context → high cost, slow responses.
- ✗ Weak or unclear prompts → vague or wrong answers.
- ✗ Ignoring stop_reason → assuming answer is complete.
- ✗ Not handling retries → random user facing failures.
- ✗ Exposing API keys in client → security risk.
- ✗ Relying on exact wording → small change in prompt breaks output.
- ✗ Not testing edge cases → hallucinations or incorrect data.

Pro Checklist

- ✓ Clear system + user prompt.
- ✓ Right model for the task.
- ✓ Limit context and tokens.
- ✓ Handle errors & retries.
- ✓ Stream for long output.
- ✓ Log everything.
- ✓ Monitor cost & performance.
- ✓ Keep improving prompts!

☆ Good prompts + Good handling = Great results!
Claude is powerful, but your integration makes it magic. ☺

Next up: Real World Projects →

9. Tool Use / Function Calling

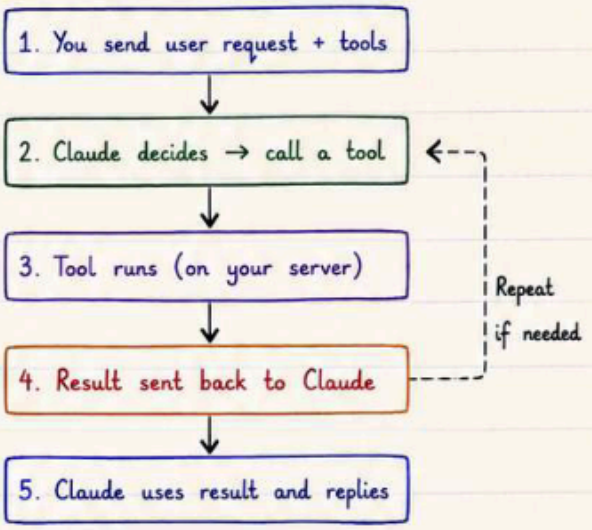
@aman_views

Claude can call tools (functions) to perform actions like search, get data, run code, etc.

What is Tool Use?

You provide a list of tools (functions) with names, descriptions and input schemas. Claude decides when to call them and with what arguments.

How it works



Example Tools (JSON)

```
[
  {
    "name": "get_weather",
    "description": "Get current weather for a city",
    "input_schema": {
      "type": "object",
      "properties": {
        "city": { "type": "string", "description": "City name" }
      },
      "required": ["city"]
    }
  },
  {
    "name": "calculator",
    "description": "Do basic math operations",
    "input_schema": {
      "type": "object",
      "properties": { "expression": { "type": "string" } },
      "required": ["expression"]
    }
  }
]
```

Sample Flow

User: What's the weather in Delhi today?

Claude: (decides to call get_weather tool)

Tool Call:

```
{ "name": "get_weather", "arguments": {
                    "city": "Delhi" } }
```

Tool Result (from your server):

```
{ "temp": "32°C", "condition": "Sunny" }
```

Claude: Today in Delhi, it's 32°C and sunny.

Request with Tools (example)

@aman_views

```
{
  "model": "claude-3-5-sonnet-20241022",
  "messages": [
  - { "role": "user", "content": "What's the weather in Delhi today?" },
  "tools": [ /* tools JSON from above */ ],
  "tool_choice": "auto"
}
```

tool_choice:

- "auto" → Claude decides
- "none" → Don't use tools
- { "type": "tool", "name": "get_weather" } → Force a specific tool

Best Practices

- Write clear tool descriptions — helps Claude pick the right tool.
- Keep input schema simple and strict.
- Handle errors in your tool and return clear messages.
- Make tools idempotent (safe to call multiple times).
- Set proper timeouts for tool execution.
- Log all tool calls and results for debugging.

Pro Tip

Tools = Superpowers ⚡

Use them wisely. Give Claude the right tools and it becomes 10x more useful!

10. Vision & Other Built-in Tools

Claude can understand images and use other built-in capabilities like code interpreter (calculations, files, etc.).

@aman_views

Vision / Image Understanding

Claude can analyze images you send and answer questions about them.

How to send image

- Send image as base64 (recommended)
- Provide a clear question or instruction

What can Claude do with images?

- ✓ Describe the image in detail
- ✓ Extract text (OCR)
- ✓ Read charts, graphs, tables
- ✓ Understand diagrams, flowcharts
- ✓ Count objects, colors, shapes
- ✓ Compare images

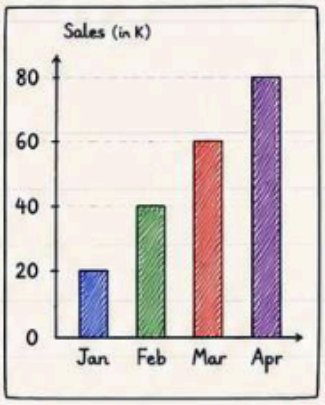
... and much more!

Example Request (Vision)

```
{
  "model": "claude-3-5-sonnet-20241022",
  "messages": [
    {
      "role": "user",
      "content": [
        { "type": "text", "text": "What's in this image?"},
        {
          "type": "image",
          "source": {
            "type": "base64",
            "media_type": "image/jpeg",
            "data": "/9j/4AAQSkZJRgABAQAAAQABAAD..."
          }
        }
      ]
    }
  ]
}
```

☆ Supports JPG, PNG, GIF, WebP (base64 encoded)

Example: Chart Analysis



→

Claude's Response:

The bar chart shows sales data (in thousands) for four months.

- Jan: 20K
- Feb: 40K
- Mar: 60K
- Apr: 80K

Sales are increasing steadily each month.

Other Built-in Tools

Tool	What it does	Example Use Cases
Code Interpreter	Run code, do math, analyze data	Calculations, data analysis, CSV/Excel, charts, file processing
File Handling	Upload & read files	Summarize PDFs, extract data
Web Search (Beta)	Get real-time info from the web	Latest news, prices, docs, etc.
System Tools	Date/time, timezone, randomness, etc.	Timestamps, scheduling, localization

Code Interpreter Example

User: What is the average of 10, 20, 30, 40, 50?

Claude (using code interpreter):

```
nums = [10, 20, 30, 40, 50]
avg = sum(nums) / len(nums)
print(avg)
```

Answer: 30

Vision & Tools – Pro Tips

- ☆ Be specific in your questions.
- ☆ For images, give context if needed.
- ☆ Keep images clear and high quality.
- ☆ Use code interpreter for heavy calculations.
- ☆ Combine tools + prompts for powerful results.

Pro Tip

Give Claude the right tool + clear instructions, it becomes your all-in-one AI assistant!

Prompt + Tool + Context = Magic ✨